Intern_Day1_Dip

March 12, 2026

1 Zakipoint Health Internship – Day 1

Name: Dipendra Raj Bhatt

1.1 1. Python Programming

1.1.1 1.0.1 Python Fundamentals

- Variables
- Loops
- Functions
- Methods

1.1.2 1.2 Logical Thinking & Coding Problems

- Practice solving algorithmic problems
- Break down problems into smaller steps

1.1.3 1.3 Object-Oriented Programming (OOP)

- Classes
- Inheritance
- Polymorphism
- Magic Methods

1.1.4 1.4 Decorators and Generators

- Learn function decorators
- Use generators for memory-efficient loops

1.1.5 1.5 Exception Handling & Logging

- Try/Except blocks
- Logging best practices

1.1.6 1.6 File Handling

- Reading & writing files
- Working with CSV, JSON

1.1.7 1.7 Writing Clean, Modular Code

- Functions and modules
- Code readability and PEP8

[1]:
```
#1.1 Python Fundamentals
##Variables
name = "Deep"        # string
age = 25              # integer
height = 5.6          # float
is_student = True     # boolean

print(name, age, height, is_student)
```

```
Deep 25 5.6 True
```

1.2 Loops

1.2.1 Main Loops

1. For Loop - Iterates over a sequence (list, string, tuple, etc.) - Syntax: `for item in sequence:` - Can use `range()` for numeric loops

2. While Loop - Repeats as long as a condition is `True` - Syntax: `while condition:` - Can cause infinite loops if condition never becomes False

3. Nested Loop - A loop inside another loop - Inner loop runs fully for each iteration of outer loop - Useful for 2D structures (lists of lists, matrices)

1.2.2 Loop Control Statements

4. break - Exits the loop immediately - Useful to stop on a condition - Skips remaining code in current loop iteration

5. continue - Skips rest of current iteration, goes to next - Useful to skip unwanted cases - Does not terminate the loop

6. else clause on loops - Executes if loop completes **without break** - Works with `for` and `while` loops - Useful for "not found" scenarios

7. pass - Does nothing; placeholder statement - Useful for minimal loop body or function stub - Prevents syntax errors when code is required

---

1.2.3 Conditional Statements

8. Conditional statements (if) - Executes code only if condition is `True` - Can use `elif` and `else` for multiple cases - Supports logical operators (`and`, `or`, `not`)

9. Sequence generator (range()) - Generates a sequence of numbers - Syntax: `range(start, stop, step)` - Commonly used with `for` loops

10. Pattern matching (match) - Introduced in Python 3.10 - Matches values against patterns like switch-case - Supports `case` blocks, guards, and destructuring

```
- **Guards**
- Extra conditions in a `case` using `if`
- Executes only if pattern matches AND guard condition is True

- **Destructuring**
- Extracts values directly from tuples, lists, dicts, or objects in a case.
- Lets you unpack data without extra assignments.
- Useful for grabbing specific elements or fields when matching patterns.
```

[2]:
```python
#Main Loops
# 1.For loop
# Measure some strings:
words = ['cat', 'window', 'defenestrate']
for w in words:
    print(w, len(w))
print(".......................")

# 2. while
print("While loop:")
count = 0
while count < 3:
    print("Count is:", count)
    count += 1
print("\n")

print(".......................")
# 3. Nested loop
print("Nested loop:")
for i in range(2):
    for j in range(2):
        print(f"i={i}, j={j}")
print("\n")
print(".......................")
# Loop control statements¶
#break
for n in range(0,10):
    for m in range(2,n):
        if n%m == 0:
```

```
            print(f"{n} equals {m}* {n//m}")
            break
print(".......................")

#continue
for num in range(2, 10):
    if num % 2 == 0:
        print(f"Found an even number {num}")
        continue
    print(f"Found an odd number {num}")

print(".......................")
print('\n')
# if-else

for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            print(n, 'equals', x, '*', n//x)
            break
    else:
        # loop fell through without finding a factor
        print(n, 'is a prime number')
print(".......................")
print('\n')
# pass
#1.loops
for i in range(5):
    if i % 2 == 0:
        pass  # Do nothing for even numbers
    else:
        print(i, "is odd")
#2.function
def my_function():
    pass  # Will implement later

my_function()  # Runs, does nothing

#3. class
class MyClass:
    pass  # Class body is empty for now

print(".......................")
print('\n')


# conditional statements
```

```
#match
def http_error(status):
    match status:
        case 400:
            return "Bad request"
        case 404:
            return "Not found"
        case 418:
            return "I'm a teapot"
        case _:
            return "Something's wrong with the internet"
http_error(400)
```

```
cat 3
window 6
defenestrate 12
…
While loop:
Count is: 0
Count is: 1
Count is: 2


…
Nested loop:
i=0, j=0
i=0, j=1
i=1, j=0
i=1, j=1


…
4 equals 2* 2
6 equals 2* 3
8 equals 2* 4
9 equals 3* 3
…
Found an even number 2
Found an odd number 3
Found an even number 4
Found an odd number 5
Found an even number 6
Found an odd number 7
Found an even number 8
Found an odd number 9
…
```

```
2 is a prime number
3 is a prime number
4 equals 2 * 2
5 is a prime number
6 equals 2 * 3
7 is a prime number
8 equals 2 * 4
9 equals 3 * 3
…


1 is odd
3 is odd
…
```

[2]:
```
'Bad request'
```

[3]:
```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

def where_is(point):
    match point:
        case Point(x=0, y=0):
            print("Origin")
        case Point(x=0, y=y):
            print(f"Y={y}")
        case Point(x=x, y=0):
            print(f"X={x}")
        case Point():
            print("Somewhere else")
        case _:
            print("Not a point")
p =Point(5,0)

where_is(p)
```

```
X=5
```

[4]:
```
# Guards
x = 10

match x:
    case n if n > 5:
        print("Greater than 5")
```

```
    case n:
        print("5 or less")
```

```
Greater than 5
```

[5]:
```
# Destructuring
point = (3, 4)

match point:
    case (0, 0):
        print("Origin")
    case (x, y):
        print(f"x={x}, y={y}")
```

```
x=3, y=4
```